

JavaScript

Revision Handbook

Interview & Quick Revision Guide

Prepared by

Wadia Fatima

Software Engineering Student

Topics Covered

- JavaScript Fundamentals
 - Variables & Data Types
 - Execution Context
 - Hoisting & TDZ
 - Functions
 - Objects & JSON
 - ES6 Features
 - Asynchronous JavaScript
 - Promises & Fetch API
 - Debounce & Throttle
 - Interview Questions
-

"Learning JavaScript isn't about memorizing syntax — it's about understanding how it thinks."

Table of Contents

01	Introduction to JavaScript Variables & Data Types
02	Variables & Data Types Scope, Types, Coercion
03	Execution Context & Hoisting Memory Phase, TDZ
04	Functions Declarations, Arrow, Callbacks
05	Objects & JSON Properties, Methods, JSON
06	ES6+ Features Destructuring, Spread, Rest
07	Async JavaScript Event Loop, Promises, async/await
08	Performance Patterns Debounce & Throttle
09	Cheat Sheets + Interview Q&A; Top 30 Questions

Introduction to JavaScript

■ What is JavaScript?

JavaScript is a **lightweight, interpreted programming language** that runs in the browser and on servers (Node.js). It makes web pages interactive.

■ Core Features

- **Dynamic Typing** — variable types are decided at runtime
- **Single Threaded** — runs one task at a time
- **Interpreted / JIT Compiled** — modern engines compile code on the fly
- **Event Driven** — reacts to user actions and network events
- **Prototype Based** — objects can inherit from other objects

■ V8 Engine (Chrome / Node.js)

■ Your JS code is written



■ V8 Engine reads the code



■ Parser creates AST (Abstract Syntax Tree)



■ Ignition Interpreter converts to bytecode



■ TurboFan JIT Compiler optimizes hot code



■ Machine Code runs on CPU

■ Interpreted vs JIT Compiled

	Interpreted	JIT Compiled
Speed	Slower	Faster
How	Line by line	Compiled at runtime
Example	Old JS engines	V8, SpiderMonkey

■ Why is JS Single Threaded?

JavaScript was designed to run in the browser. Browsers need to manipulate the DOM. If two threads changed the DOM at the same time — **chaos**.

So JavaScript uses **one thread** with an **event loop** to handle async tasks without blocking.

■ Synchronous vs Asynchronous

Synchronous	Asynchronous
Runs line by line	Doesn't wait — continues
Blocks next line	Uses callbacks/promises
Default JS behavior	Needs Web APIs / Event Loop
alert(), console.log()	setTimeout, fetch, addEventListener

■ Quick Revision

JS = Single Threaded + Synchronous by default. Async magic comes from the **Browser / Node.js environment** — not JS itself.

■ Interview Tip

Interviewers love asking: *'Is JavaScript synchronous or asynchronous?'* Answer: JS is synchronous and single-threaded. Async behavior is provided by the browser via Web APIs.

Variables & Data Types

■ var / let / const

Feature	var	let	const
Scope	Function	Block	Block
Re-declare	■ Yes	■ No	■ No
Re-assign	■ Yes	■ Yes	■ No
Hoisting	■ (undefined)	■ (TDZ)	■ (TDZ)
Use Today	■ Avoid	■ Use	■ Prefer

```
var x = 10; // function scope, can redeclare
let y = 20; // block scope, can reassign
const z = 30; // block scope, cannot reassign

// const with objects – the REFERENCE is locked, not content
const user = { name: 'Ali' };
user.name = 'Sara'; // ■ allowed!
user = {}; // ■ Error – can't reassign
```

■ Scope

Scope Type	Where Accessible	Keyword
Global	Everywhere	var (outside function)
Function	Inside function only	var, let, const
Block	Inside { } only	let, const

■ Data Types

Category	Types	Example
Primitive	String, Number, Boolean, Null, Undefined, Symbol, BigInt	"hello", 42, true, null
Non-Primitive	Object, Array, Function	{ }, [], function(){ }

Memory Trick — Primitives store **value**. Non-Primitives store **reference**.

Comparing two objects with `==` will return **false** even if content looks same — because they point to different memory locations.

■ Type Coercion & `==` vs `===`

Type Coercion = JS automatically converts types when comparing or operating.

```
// == (loose equality) – converts types first
0 == false // true (0 is falsy)
'' == false // true
1 == '1' // true (string '1' converted to number)

// === (strict equality) – no conversion
1 === '1' // false (different types)
0 === false // false
```

■ Truthy & Falsy

Falsy Values	Truthy Values
false, 0, "", null, undefined, NaN	Everything else
"" (empty string)	"hello", 1, [], {}, -1

■ ■ typeof null

typeof null === "object"

This is a historical bug in JavaScript — null is a primitive, but typeof returns "object". Be aware of this in interviews.

■ Interview Tip

Always use `===` in real code. Interviewers ask: *'What is the difference between == and ==='*. Answer: `==` checks value with type coercion, `===` checks value AND type.

Execution Context & Hoisting

■ What is Execution Context?

Every time JavaScript runs code, it creates an **Execution Context** — a workspace where code is prepared and executed.

Type	Created When	How Many
Global Execution Context	Script first runs	1 (always)
Function Execution Context	Each function call	1 per function call

■ Two Phases of Execution

■ ■ PHASE 1 — Memory (Creation) Phase

↓

■ All variables → allocated as undefined

↓

■ All functions → stored in memory completely

↓

■ ■ ■ PHASE 2 — Code Execution Phase

↓

■ Code runs line by line

↓

■ Values are assigned

↓

■ Functions are called

■ Hoisting

Hoisting = JS moves variable and function **declarations** to the top of their scope during the Memory Phase.

```
// Function hoisting – works perfectly
sayHi(); // ■ 'Hello!'

function sayHi() { console.log('Hello!'); }

// var hoisting – declaration moves up, but NOT value
console.log(a); // undefined (not error!)
```

```
var a = 5;

// let/const — hoisted but NOT initialized (TDZ!)
console.log(b); // ■ ReferenceError

let b = 10;
```

■ Temporal Dead Zone (TDZ)

The TDZ is the period between when **let/const** is hoisted and when it's actually initialized.

During this time — you **cannot access** the variable. Trying to will throw a **ReferenceError**.

■ Memory Phase — let/const is hoisted (memory allocated)

↓

■ TDZ BEGINS — variable exists but is not initialized

↓

■ Code reaches the declaration line

↓

■ TDZ ENDS — variable is now initialized

↓

■ Variable can be accessed safely

■ Call Stack

The **Call Stack** is where JS keeps track of which function is currently running.

```
function a() { b(); }
function b() { c(); }
function c() { console.log('done'); }

a();

// Call Stack: [a → b → c] then pops: c → b → a → empty
```

■ Interview Tip

Very common Q: 'What is hoisting?' **Answer:** Hoisting is when JS moves declarations to the top of scope during memory phase. Functions are fully hoisted. var is hoisted as undefined. let/const are in TDZ.

■ Quick Revision

Execution Context → Memory Phase (hoist) → Code Phase (execute) | TDZ → let/const hoisted but NOT accessible

Functions

Type	Syntax	Hoisted?	Has 'this'?
Declaration	function foo() {}	■ Yes	■ Yes
Expression	const foo = function() {}	■ No	■ Yes
Arrow Function	const foo = () => {}	■ No	■ No (uses outer)
Anonymous	function() {}	—	■ Yes

■ Function Declaration

```
function greet(name) {
  return 'Hello, ' + name;
}

console.log(greet('Ali')); // 'Hello, Ali'

// ■ Can be called BEFORE the declaration (hoisted)
greet('Sara'); // works!
```

■ Function Expression

```
const greet = function(name) {
  return 'Hello, ' + name;
};

// ■ Cannot call before declaration
greet('Ali'); // Error if called before this line
```

■ Arrow Function

```
// Traditional
const add = function(a, b) { return a + b; };

// Arrow – shorter
const add = (a, b) => a + b;

// Single param – no parentheses needed
const double = n => n * 2;
```

```
// ■■ Arrow functions do NOT have their own 'this'
// They use 'this' from the surrounding scope
```

■ Callback Function

A **callback** is a function passed as an argument to another function — called later.

```
function doTask(callback) {
  console.log('Task done');
  callback(); // call the function we received
}

doTask(function() {
  console.log('Callback called!');
});

// Real world: setTimeout, map, filter, forEach
[1,2,3].forEach(n => console.log(n)); // callback!
```

■ Higher Order Function (HOF)

A **Higher Order Function** either **takes a function as input** OR **returns a function**.

```
// Takes a function – Higher Order Function
[1,2,3].map(n => n * 2); // map is a HOF

// Returns a function
function multiplier(x) {
  return function(n) { return n * x; };
}

const triple = multiplier(3);
triple(5); // 15
```

■ Interview Tip

'What is a callback?' — A function passed into another function to be called later. Example: `setTimeout` takes a callback. `Array.map` takes a callback.

■ Remember

Arrow functions: shorter syntax + no own 'this'. Perfect for callbacks. HOF: `map`, `filter`, `reduce` are all Higher Order Functions.

Objects & JSON

■ Objects

An object is a **collection of key-value pairs**. Keys are strings, values can be anything.

```
const user = {
  name: 'Ali', // property
  age: 25,
  greet: function() { // method
    console.log('Hi, I am ' + this.name);
  }
};

// Access
user.name; // dot notation → 'Ali'
user['name']; // bracket notation → 'Ali'

// Call method
user.greet(); // 'Hi, I am Ali'
```

■ Useful Object Methods

Method	What it does	Returns
Object.keys(obj)	Get all keys	Array of keys
Object.values(obj)	Get all values	Array of values
Object.entries(obj)	Get key-value pairs	Array of [key, value]
Object.assign()	Copy/merge objects	New object
{...spread}	Spread into new obj	New object

```
const user = { name: 'Ali', age: 25 };

Object.keys(user); // ['name', 'age']
Object.values(user); // ['Ali', 25]
Object.entries(user); // [['name', 'Ali'], ['age', 25]]

// Spread to copy
const copy = { ...user, city: 'Lahore' };
```

```
// { name: 'Ali', age: 25, city: 'Lahore' }
```

■ JSON (JavaScript Object Notation)

JSON is a **text format** to store and transfer data. It looks like a JS object but is a **string**.

JS Object	JSON
Keys without quotes OK	Keys MUST be in double quotes
Functions allowed	No functions
Undefined allowed	No undefined
Type: Object	Type: String

```
const obj = { name: 'Ali', age: 25 };

// JS Object → JSON string
const json = JSON.stringify(obj);
// '{"name":"Ali","age":25}'

// JSON string → JS Object
const parsed = JSON.parse(json);
parsed.name; // 'Ali'
```

■ Interview Tip

'Difference between JS object and JSON?' — JSON is just a string format. JS object is actual data in memory. Use `JSON.stringify()` to convert object to JSON, `JSON.parse()` to convert back.

ES6+ Features

■ Template Literals

```
const name = 'Ali';
const age = 25;

// Old way (messy)
console.log('Hello ' + name + ', you are ' + age);

// Template Literal (clean!)
console.log(`Hello ${name}, you are ${age}`);

// Multi-line strings
const msg = `Line 1
Line 2`;
```

■ Destructuring

```
// Array Destructuring
const [a, b, c] = [1, 2, 3];
console.log(a); // 1

// Object Destructuring
const { name, age } = { name: 'Ali', age: 25 };
console.log(name); // 'Ali'

// Rename while destructuring
const { name: userName } = user;
console.log(userName); // 'Ali'

// Default value
const { city = 'Lahore' } = user;
```

■ Spread vs Rest

	Spread (...)	Rest (...)
Meaning	Expand / spread out	Collect / gather in
Used for	Copying, merging	Function parameters

Position	In call / literal	In function definition
----------	-------------------	------------------------

```
// SPREAD – expand an array
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1,2,3,4,5]

// SPREAD – merge objects
const merged = { ...user, city: 'PK' };

// REST – collect remaining args
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}

sum(1, 2, 3, 4); // 10
```

Memory Trick

Spread → Expand & copy out

Rest → Collect & gather in

■ Default Parameters

```
function greet(name = 'Guest') {
  return `Hello, ${name}`;
}

greet('Ali'); // 'Hello, Ali'
greet(); // 'Hello, Guest' ← uses default
```

■ Optional Chaining ?.

Safely access nested properties. Returns **undefined** instead of throwing an error if property doesn't exist.

```
const user = { address: { city: 'Lahore' } };

// Old way (risky)
user.address.city // works
user.phone.number // ■ TypeError!

// Optional Chaining (safe)
user?.address?.city // 'Lahore'
user?.phone?.number // undefined (no error!)
```

■ Interview Tip

Spread = expand. Rest = collect. Same symbol (...) but different context. Optional chaining is often asked with null checks.

Asynchronous JavaScript

■ Core Concept to Remember

JavaScript itself is synchronous.

The browser provides asynchronous behavior through Web APIs.

"JavaScript doesn't own a clock — it asks the browser to keep track of time."

■ The Full Picture — How Async Works

■ Your JS code calls `setTimeout()` / `fetch()` / `addEventListener()`

↓

■ JS hands task to Browser Web APIs (not JS's job anymore)

↓

■ JS continues running rest of code (non-blocking!)

↓

■ Browser finishes task (timer done / data received)

↓

■ Callback is placed in Task Queue (or Microtask Queue)

↓

■ Event Loop checks: Is Call Stack empty?

↓

■ If YES → Event Loop moves callback to Call Stack

↓

■ Callback executes

■ Call Stack

The **Call Stack** is a data structure that tracks which function is currently executing. It works like a stack of plates — last in, first out (LIFO).

Call Stack Rule	Detail
LIFO	Last In, First Out
Sync only	Only synchronous code here

■ Web APIs (Browser's Toolbox)

These are **NOT** part of JavaScript. The browser provides them:

- `setTimeout` / `setInterval` — timer functions
- `fetch()` — HTTP requests
- `addEventListener` — DOM events
- `XMLHttpRequest` — older HTTP requests
- `Geolocation`, `localStorage` — device APIs

■ Task Queue vs Microtask Queue

	Microtask Queue	Task Queue (Macro)
Priority	■ Higher (runs first)	■ Lower
Examples	Promise .then(), queueMicrotask()	setTimeout, setInterval, fetch callback
When runs	After each task, before next task	After microtasks are empty

```
console.log('1');

setTimeout(() => console.log('2'), 0); // Task Queue

Promise.resolve().then(() => console.log('3')); // Microtask

console.log('4');

// Output: 1 → 4 → 3 → 2
// Microtask (3) runs before setTimeout (2) even with 0ms!
```

■ Why setTimeout(0) Doesn't Run Immediately

`setTimeout(fn, 0)` does NOT mean "run right now."

It means: "put this in the **Task Queue** after 0ms." The callback still has to wait for:

1. Call Stack to be completely empty
2. All Microtasks (Promises) to finish
3. Then the Event Loop picks it up

■ Promises

A **Promise** represents a value that may be available now, in the future, or never.

State	Meaning	When
■ Pending	Waiting...	Initial state
■ Fulfilled	Success!	resolve() called
■ Rejected	Failed!	reject() called

```
const promise = new Promise((resolve, reject) => {
  const success = true;
  if (success) resolve('Done!');
  else reject('Error!');
});

promise
  .then(result => console.log(result)) // 'Done!'
  .catch(err => console.log(err)) // 'Error!'
  .finally(() => console.log('Always runs'));
```

■ Promise Chaining

```
fetch('https://api.example.com/users')
  .then(response => response.json()) // convert to JSON
  .then(data => console.log(data)) // use the data
  .catch(error => console.log(error));
```

■ async / await

async/await is syntactic sugar over Promises. Makes async code look synchronous and clean.

Promise Chain	async/await
fetch().then().catch()	await fetch() inside try/catch
Harder to read with nesting	Clean, reads top to bottom
Both use Microtask Queue	Both use Microtask Queue

```
// async function always returns a Promise
async function getUser() {
  try {
    const response = await fetch('/api/user'); // wait here
```

```
const data = await response.json(); // wait here
console.log(data);
} catch (error) {
  console.log('Error:', error);
}
}

getUser(); // call it – returns a Promise
```

■ Remember

async functions always return a Promise — even if you return a plain value, it gets wrapped in a resolved Promise.

■ setTimeout & setInterval

```
// setTimeout – runs ONCE after delay
setTimeout(() => console.log('After 2s'), 2000);

// setInterval – runs REPEATEDLY
const id = setInterval(() => console.log('tick'), 1000);

// Stop interval
clearInterval(id);

// Stop timeout (if not yet run)
const tid = setTimeout(...);
clearTimeout(tid);
```

■ Event Listeners

```
const btn = document.getElementById('myBtn');

// Add listener
btn.addEventListener('click', function(event) {
  console.log('Clicked!', event.target);
});

// ■■ Event listeners use Browser Web APIs
// The callback goes to Task Queue when event fires
// Event Loop moves it to Call Stack when stack is empty
```

■ Interview Tip

'What is the Event Loop?' — The Event Loop constantly checks if the Call Stack is empty. When empty, it takes the next callback from the Microtask Queue (first) or Task Queue (second) and pushes it to the Call Stack.

■ Quick Revision

Call Stack → runs sync code | Web APIs → handles async | Task Queue → setTimeout callbacks | Microtask Queue → Promise callbacks | Event Loop → connects them all

Performance Patterns

■ Debounce

Debounce = Wait until the user **stops** doing something, then run.

■ Memory Trick

Debounce = DELAY. Like waiting for someone to stop talking before you reply.

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer); // reset timer every call
    timer = setTimeout(() => {
      fn.apply(this, args); // only runs after delay
    }, delay);
  };
}

// Real world: search input
const handleSearch = debounce((e) => {
  fetch('/api/search?q=' + e.target.value);
}, 500);

searchInput.addEventListener('input', handleSearch);
// API called only 500ms after user STOPS typing
```

■ Throttle

Throttle = Run at most **once every X milliseconds**, no matter how many times triggered.

■ Memory Trick

Throttle = LIMIT. Like a speed limit — you can't go faster than allowed.

```
function throttle(fn, limit) {
  let lastRun = 0;
  return function(...args) {
    const now = Date.now();
    if (now - lastRun >= limit) {
```

```

lastRun = now;
fn.apply(this, args);
}
};
}

// Real world: scroll event
const handleScroll = throttle(() => {
  console.log('scroll position:', window.scrollY);
}, 200);

window.addEventListener('scroll', handleScroll);

// Runs max once every 200ms even if scrolling fast

```

■ Debounce vs Throttle — Comparison

	Debounce	Throttle
Idea	Wait for pause	Limit frequency
Runs when	After user STOPS	Every X ms while active
Use case	Search box typing	Scroll, resize, mousemove
Example	API call after 500ms idle	Update UI max once/200ms
Risk	May never run if always active	May miss last event

■ Real World Use Cases

Event	Best Pattern	Why
Search box input	Debounce	Wait until typing stops
Window scroll	Throttle	Smooth, limited updates
Window resize	Throttle	Layout recalc every Xms
Button click submit	Debounce	Prevent double submit
API autocomplete	Debounce	Reduce API calls
Mouse move tracking	Throttle	Don't fire 100x/sec

■ Interview Tip

'What's the difference between debounce and throttle?' Debounce waits until activity stops then fires once. Throttle fires at most once per time interval regardless. Both are performance optimizations.

Final Cheat Sheet

Execution Context	A workspace where JS runs code	Memory Phase	Variables hoisted, functions stored
Code Phase	Code runs line by line	Hoisting	Declarations moved to top
TDZ	let/const hoisted but not accessible	Call Stack	Tracks which function is running
Web APIs	Browser handles timers, fetch, events	Task Queue	setTimeout / setInterval callbacks wait here
Microtask Queue	Promise .then() callbacks — run first	Event Loop	Moves callbacks to Call Stack when empty
Promise	Future value — pending/fulfilled/rejected	async/await	Clean way to write async code
Spread ...	Expand arrays or objects	Rest ...	Collect multiple args into array
Destructuring	Extract values from arrays/objects	Debounce	Wait — run after user stops
Throttle	Limit — run max once per interval	Closure	Function that remembers outer variables
HOF	Function that takes/returns a function	Callback	Function passed as argument, called later
== vs ===	Loose equality vs strict equality	var vs let/const	Function scope vs Block scope
JSON.stringify	JS Object → JSON string	JSON.parse	JSON string → JS Object
Optional Chaining	?. — safe property access		

Top 30 Interview Questions

01. What is JavaScript?

JavaScript is a lightweight, interpreted, dynamic programming language mainly used to make web pages interactive. It runs in browsers and on servers via Node.js.

02. Is JavaScript synchronous or asynchronous?

JavaScript is synchronous and single-threaded by default. Asynchronous behavior (`setTimeout`, `fetch`) is provided by the browser through Web APIs and the Event Loop.

03. What is the difference between `var`, `let`, and `const`?

`var` is function-scoped and can be re-declared. `let` is block-scoped and can be reassigned. `const` is block-scoped and cannot be reassigned. Prefer `const`, then `let`. Avoid `var`.

04. What is hoisting?

Hoisting is when JS moves declarations to the top of their scope during the Memory Phase. Function declarations are fully hoisted. `var` is hoisted as `undefined`. `let/const` are in TDZ.

05. What is the Temporal Dead Zone (TDZ)?

The TDZ is the period between when `let/const` is hoisted and when it's initialized. Accessing a variable in TDZ throws a `ReferenceError`.

06. What is the Event Loop?

The Event Loop constantly monitors the Call Stack. When the stack is empty, it moves callbacks from the Microtask Queue (first) or Task Queue (second) into the Call Stack.

07. What is the difference between `==` and `===`?

`==` checks value with type coercion (converts types first). `===` checks both value AND type — no conversion. Always prefer `===`.

08. What is a closure?

A closure is when a function remembers variables from its outer scope even after that outer function has finished executing. Used in data privacy and factory functions.

09. What is the difference between `null` and `undefined`?

`undefined` means a variable was declared but not assigned. `null` is intentionally assigned to mean 'no value'. `typeof null` is `'object'` (a historical bug in JS).

10. What is a Promise?

A Promise is an object representing a future value. It can be pending, fulfilled, or rejected. Use `.then()` for success, `.catch()` for errors, `.finally()` for cleanup.

11. What is `async/await`?

`async/await` is syntactic sugar over Promises. An `async` function always returns a Promise. `await` pauses execution until the Promise resolves. Use `try/catch` for errors.

12. What is the difference between `setTimeout` and `setInterval`?

`setTimeout` runs a callback ONCE after a delay. `setInterval` runs it REPEATEDLY at every interval. Use `clearTimeout()` and `clearInterval()` to cancel them.

13. Why does `setTimeout(fn, 0)` not execute immediately?

`setTimeout(fn, 0)` puts the callback in the Task Queue after 0ms. It still waits for the Call Stack to be empty and all Microtasks (Promises) to finish first.

14. What is the difference between Microtask Queue and Task Queue?

Microtask Queue has higher priority — it runs after each task (Promise `.then`). Task Queue has lower priority — `setTimeout`, `setInterval` callbacks go here.

15. What is the difference between spread and rest?

Both use `...` but differently. Spread expands an array or object. Rest collects multiple arguments into an array. Spread is in function calls, rest is in function definitions.

16. What is destructuring?

Destructuring extracts values from arrays or objects into variables. `const {name, age} = user;` or `const [a, b] = arr;`. You can also rename and set defaults.

17. What is optional chaining (`?.`)?

Optional chaining safely accesses nested properties. If a property doesn't exist, it returns `undefined` instead of throwing a `TypeError`. Example: `user?.address?.city`

18. What is a callback function?

A callback is a function passed as an argument to another function, to be called later. Example: `setTimeout(callback, 1000)` or `array.forEach(callback)`.

19. What is a Higher Order Function?

A function that takes another function as an argument OR returns a function. Examples: `map`, `filter`, `reduce`, `forEach`, `setTimeout` are all higher order functions.

20. What is type coercion?

Type coercion is when JS automatically converts one data type to another. Happens with `==` operator. Example: `'5' == 5` is true because `'5'` is converted to `5`.

21. What is the difference between function declaration and function expression?

Function declarations are hoisted fully and can be called before definition. Function expressions are not hoisted. Arrow functions are a type of function expression.

22. What is JSON?

JSON (JavaScript Object Notation) is a text format for storing and transferring data. It looks like a JS object but is a string. Use `JSON.stringify()` and `JSON.parse()`.

23. What are falsy values in JavaScript?

`false`, `0`, `""` (empty string), `null`, `undefined`, `NaN` are falsy. Everything else is truthy — including empty arrays `[]` and empty objects `{}`.

24. What is the difference between `Object.keys`, `Object.values`, and `Object.entries`?

`Object.keys` returns array of keys. `Object.values` returns array of values. `Object.entries` returns array of `[key, value]` pairs. All return arrays.

25. What is debounce?

Debounce delays function execution until user STOPS triggering it for X ms. Used for search inputs to reduce unnecessary API calls. Example: wait 500ms after user stops typing.

26. What is throttle?

Throttle limits function to run at most once per X ms. Even if triggered 100 times, it fires at a controlled rate. Used for scroll, resize, mousemove events.

27. What is the call stack?

The call stack is a LIFO data structure that tracks function execution. When a function is called, it's pushed. When it returns, it's popped. Stack overflow = too many nested calls.

28. What are Web APIs?

Web APIs are features provided by the BROWSER (not JavaScript). Examples: setTimeout, fetch, localStorage, DOM, geolocation. They handle async tasks outside JS.

29. What is the difference between deep copy and shallow copy?

Shallow copy copies only the top level. Nested objects are still referenced. Use spread {...} or Object.assign for shallow. Use JSON.parse(JSON.stringify()) or structuredClone for deep copy.

30. What is scope in JavaScript?

Scope determines where a variable can be accessed. Global scope = everywhere. Function scope = inside function only (var). Block scope = inside {} only (let, const).

Final Thoughts

Learning JavaScript is not about memorizing every method or syntax.

It's about understanding how JavaScript works behind the scenes — and knowing when to use the right concepts to solve problems.

Use this handbook for revision, strengthen your fundamentals, and continue building projects.

The best way to improve is by writing code consistently.

Happy Coding!

Prepared by

Wadia Fatima